

CS486 SOLUTION - TEST 4

Based on the FINAL REVIEW structure; total: 10 points

Question 1 - ER Models

[1 pt]

Solution. Seller key SID; Customer key CID; Order key OID. Product is weak/owner-qualified under Seller with key (SID,PCode). OrderItem is the associative entity between Order and Product and stores Qty/SalePrice; because the same product cannot repeat in one order, key is (OID,SID,PCode). Cardinalities: Customer 1:N Orders; Orders M:N Product resolved by OrderItem; Seller 1:N Product.

Question 2 - Query Languages

[2 pt]

Solution. (a) Sellers with at least one sold item minus sellers with any above-list sale:

$$\pi_{SID}(OrderItem) - \pi_{oi.SID}(\sigma_{oi.SID=p.SID \wedge oi.PCode=p.PCode \wedge oi.SalePrice > p.ListPrice}(\rho_{oi}(OrderItem) \times \rho_p(Product))).$$

(b)

$$\{o \mid Orders(o) \wedge \exists i_1 \exists i_2 (OrderItem(i_1) \wedge OrderItem(i_2) \wedge i_1.OID = o.OID \wedge i_2.OID = o.OID \wedge i_1.SID \neq i_2.SID)\}.$$

(c)

```
WITH s AS (
  SELECT SID,PCode,SUM(Qty) qty
  FROM OrderItem GROUP BY SID,PCode
), r AS (
  SELECT *,DENSE_RANK() OVER(PARTITION BY SID ORDER BY qty DESC) rk
  FROM s
)
SELECT SID,PCode,qty FROM r WHERE rk=1;
```

Question 3 - Database Constraints

[1 pt]

Solution. Use CHECK(Qty > 0) and CHECK(SalePrice >= 0). For suspension, forbid any item whose order date is on/after the seller's suspension date:

$$\neg \exists i \exists o \exists s (OrderItem(i) \wedge Orders(o) \wedge SellerStatus(s) \wedge i.OID = o.OID \wedge i.SID = s.SID \wedge s.SuspendedFrom \leq o.ODate)$$

Implement with an INSERT/UPDATE trigger on OrderItem (and, to be fully robust, relevant updates of Orders/SellerStatus) that joins the affected rows and raises an error when the condition exists.

Question 4 - Serialization & Concurrency

[2 pt]

Solution. P gives $T_1 \rightarrow T_2$. Q gives $T_2 \rightarrow T_3, T_3 \rightarrow T_1$ (and $T_2 \rightarrow T_1$ because $w_2(Q)$ precedes $r_1(Q)$). R gives $T_1 \rightarrow T_3, T_1 \rightarrow T_2$, and $T_3 \rightarrow T_2$. There is $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$, so not conflict-serializable.

If T1 reads Q from T3, recoverability requires T3 commit before T1. Given c_1, c_3, c_2 , T1 commits first, so not recoverable. It is not cascadeless because T1 reads an uncommitted T3 value.

Timestamp deadlock prevention: wait-die allows older to wait and forces younger requester to abort when requesting a lock held by older; wound-wait lets older preempt/abort younger and makes younger wait for older. Both impose acyclic age-based waiting, preventing deadlock cycles.

Question 5 - Recovery**[1 pt]**

Solution. Winners: T2 and T1. Losers: T3 and T4. REDO committed updates: T2 $Y := 12$; T1 $X := 8$ then $X := 11$ (or simply ensure final $X=11$ in logical summary). UNDO losers in reverse log order: T4 restore $W := 1$; T3 restore $Z := 4$. The checkpoint limits where recovery analysis begins, subject to the protocol's active-transaction list.

Question 6 - Functional Dependencies & Normalization**[1 pt]**

Solution. A^+ gives A,B,C and AC gives D, then D gives E; therefore A is a candidate key.

Because the only candidate key is the single attribute A, 2NF holds automatically. $B \rightarrow C$ has B not a superkey and C nonprime, so 3NF fails. Highest normal form is 2NF.

One BCNF-style sequence: decompose on $B \rightarrow C$ into $R_1(B, C)$ and $R'(A, B, D, E)$. In R' , $D \rightarrow E$ violates BCNF, so decompose into $R_2(D, E)$ and $R_3(A, B, D)$. Each binary decomposition is lossless because the common determinant functionally determines one component. Dependency $AC \rightarrow D$ is not preserved directly after this decomposition, so a dependency-preserving 3NF synthesis may be preferred if preservation is required.

Question 7 - Concurrency Control Execution**[1 pt]**

Solution. Both read $Q=10$ before either writes. T1 approves 7 units and computes 3; T2 also approves 5 units and computes 5. If T1 writes 3 then T2 writes 5, final stored $Q=5$; if the writes reverse, final $Q=3$. In either case 12 units were promised from only 10. This is a lost-update/check-then-act anomaly: the final field value alone does not encode the two external sales decisions, so the business invariant can be violated even if the stored quantity is nonnegative.

Question 8 - Query Optimization**[1 pt]**

Solution. One seller: clustered composite B+ tree supports the prefix SID range. Cost $\approx 4 + 800 = 804$ page I/Os, far below 100,000-page scan.

One order: hash on OID is ideal for equality. Expected cost is a few bucket/index I/Os plus up to about 8 scattered data-page fetches, roughly 10–12 I/Os, far below a full scan. The (SID, PCode) B+ tree is not keyed by OID and does not directly help.